



UNIVERSIDADE ESTADUAL DA PARAÍBA
CIÊNCIAS DA COMPUTAÇÃO

DAVID EMMANUEL XAVIER DOS SANTOS
GABRIEL TRIGUEIRO FERNANDES

**PROJETO EXPERIMENTAL: ANÁLISE COMPARATIVA DE TÉCNICAS DE PROJETO
DE ALGORITMOS**

CAMPINA GRANDE

2026

1. Introducao.....	4
1.1. Contextualização do Problema.....	4
1.2. Objetivos.....	4
1.2.1. Geral.....	4
1.2.2. Especificos.....	4
2. Metodologia e Implementação.....	4
2.1. Estrutura do Sistema Desenvolvido.....	4
2.2. Formato de Dados de Entrada e Saida.....	5
2.3. Métricas de Avaliação.....	5
3. Algoritmos Implementados.....	6
3.1. Backtracking.....	6
3.1.1. Análise de Corretude.....	6
3.1.2. Analise de Desempenho.....	6
3.1.2.1. Espacial.....	6
3.1.2.2. Temporal.....	6
3.2. Branch and Bound.....	7
3.2.1. Analise de Corretude.....	7
3.2.2. Analise de Desempenho.....	7
3.2.2.1. Espacial.....	7
3.2.2.2. Temporal.....	8
3.3. Programacao Dinamica.....	8
3.3.1. Análise de Corretude.....	8
3.3.2. Análise de Desempenho.....	9
3.3.2.1. Espacial.....	9
3.3.2.2. Temporal.....	9
3.4. Estratégia Gulosa.....	9
3.4.1. Análise de Corretude.....	10
3.4.2. Análise de Desempenho.....	10
3.4.2.1. Espacial.....	10
3.4.2.2. Temporal.....	10
4. Experimentos e Resultados.....	11
4.1. Instancias de Teste.....	11
4.2. Resultados Obtidos.....	11
Cenário 1 — input.txt (n=4, T=9).....	11
Cenário 2 — padrao.txt (n=8, T=15) — Caso Base.....	12
Cenário 3 — solucao_imediata.txt (n=10, T=12) — Vantagem BT/BB.....	12
Cenário 4 — sem_solucao.txt (n=12, T=79) — Sem Solução.....	12
Cenário 5 — poda_maxima.txt (n=10, T=5) — Maxima Poda.....	13

Cenario 6 — solucao_no_final.txt (n=10, T=9) — Destaque da DP.....	13
Cenario 7 — fila_grande_bb.txt (n=15, T=30) — Ponto Fraco do BB.....	13
Tabela Consolidada — Todos os Cenários e Algoritmos.....	14
5. Análise e Discussão.....	15
5.1. Comparacao das Estrategias.....	15
6. Conclusao.....	16
7. Referencias.....	18

1. Introdução

1.1. Contextualização do Problema

O Problema do Subconjunto Soma (Subset Sum) consiste em, dado um conjunto de n inteiros positivos e um valor alvo T , determinar se existe algum subconjunto cujos elementos somam exatamente T .

Formalmente: dado $S = \{a_1, a_2, \dots, a_n\}$ e T , existe $I \subseteq \{1, \dots, n\}$ tal que $\text{soma}(a_i \text{ para } i \text{ em } I) = T$?

O problema é NP-completo. Não se conhece algoritmo de tempo polinomial no caso geral, e acredita-se que nenhum exista. O espaço de busca possui 2^n subconjuntos possíveis, o que torna a exploração exaustiva inviável para entradas grandes.

Este relatório analisa e compara quatro técnicas clássicas de projeto de algoritmos aplicadas ao Subset Sum, avaliando como cada uma trata o espaço de busca exponencial.

1.2. Objetivos

1.2.1. Geral

Implementar, analisar e comparar formalmente as técnicas de Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa aplicadas ao Problema do Subconjunto Soma, avaliando empiricamente suas características de tempo, memória, qualidade da solução e escalabilidade.

1.2.2. Específicos

- Implementar os quatro algoritmos em Python com coleta de métricas padronizadas.
- Projetar cenários de teste que explorem os pontos fortes e fracos de cada técnica.
- Medir empiricamente: tempo de execução, pico de memória, nós explorados e podas realizadas.
- Analisar a complexidade assintótica teórica de cada algoritmo.
- Discutir quando cada técnica é a mais adequada para o problema.

2. Metodologia e Implementação

2.1. Estrutura do Sistema Desenvolvido

O sistema foi desenvolvido em Python 3.11 com arquitetura modular. Cada algoritmo está em seu próprio módulo dentro do pacote `algorithms/`, com interface padronizada:

```
def algoritmo(nums: list[int], target: int) -> tuple[list[int] | None, dict]:  
    # retorna (indices_solucao_base1_ou_None, dicionario_de_metricas)
```

Estrutura de diretorios:

```
taal-project-subset-sum/  
  algorithms/  
    backtracking.py      # Busca DFS com poda  
    branch_bound.py      # BFS com limite superior (suffix_sum)  
    dynamic_programming.py # Tabela DP bottom-up  
    greedy.py            # Heuristica gulosa decrescente  
  examples/              # Arquivos de entrada dos cenarios de teste  
  main.py                # Menu interativo  
  io_handler.py          # Leitura de entrada (arquivo ou manual)  
  metrics.py             # Exibicao de resultados e metricas  
  README.md             # Instrucoes de execucao
```

2.2. Formato de Dados de Entrada e Saida

Entrada (arquivo .txt ou digitação manual):

```
n T  
a1 a2 a3 ... an
```

Onde n = numero de elementos e T = soma alvo.

Saida — com solução:

```
YES  
k  
i1 i2 ... ik
```

Saida — sem solução:

```
NO
```

Os índices i1..ik sao base-1 (o primeiro elemento tem índice 1).

2.3. Métricas de Avaliação

Todas as métricas são coletadas de forma idêntica para os quatro algoritmos:

Metrica	O que mede	Ferramenta
Nos visitados	Estados/células efetivamente processados	Contador interno
Podas realizadas	Ramos descartados sem exploração	Contador interno
Eficiencia de poda	podas / (visitas + podas) x 100%	Calculado
Tempo de execucao	Duração real da busca em ms	time.perf_counter()
Pico de memoria	Maximo de memoria alocada em KB	tracemalloc
Qualidade da solução	Exata (otima) ou aproximada (heurística)	Verificacao direta

Obs.: Para Programação Dinâmica, "nos visitados" = células da tabela computadas. Para Estratégia Gulosa, "podas" = elementos rejeitados por excederem o alvo restante.

3. Algoritmos Implementados

3.1. Backtracking

O Backtracking percorre uma árvore binária de decisão em profundidade (DFS). Em cada nível i , decide se o elemento $\text{nums}[i]$ será incluído ou excluído do subconjunto. Ao encontrar um ramo inviável, retorna ao nó anterior e testa o outro caminho.

3.1.1. Análise de Corretude

O Backtracking é correto e completo: sempre encontra uma solução quando ela existe e prova a impossibilidade quando não existe. A corretude decorre de:

- Explorar sistematicamente todos os 2^n subconjuntos possíveis (no pior caso).
- As podas são seguras: descartar um ramo somente quando é provado que nenhum descendente pode ser solução.
- Poda por excesso: $\text{nums}[i] > \text{remaining}$ — incluir o elemento j ultrapassaria o alvo. Nenhum subconjunto incluindo j pode ser solução.
- Poda por insuficiência: $\text{suffix_sum}[i] < \text{remaining}$ — mesmo somando todos os restantes, o alvo é inalcançável.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

O Backtracking usa apenas a pilha de recursos. A profundidade máxima é n (um nível por elemento). Cada frame da pilha armazena: `index`, `remaining`, `current` (lista de índices).

Caso	Complexidade de Espaço
Pior caso	$O(n)$ — pilha de recursão de profundidade n
Melhor caso	$O(\text{profundidade da solução})$
Memória observada nos testes	0,28 a 0,41 KB (constante na prática)

A memória é praticamente constante e independente do valor de T , pois não aloca estruturas proporcionais ao alvo.

3.1.2.2. Temporal

A complexidade temporal depende da eficácia das podas:

Caso	Complexidade de Tempo	Situacao
Pior caso	$O(2^n)$	Sem podas — explora todos os subconjuntos
Caso medio	Muito menor que $O(2^n)$	Podas eliminam grandes blocos

Melhor caso	$O(n)$	Soluç Ao encontrada no primeiro caminho DFS
-------------	--------	---

Nos testes, o pior caso observado foi o cenário `sol_no_final` (221 nós visitados para $n=10$), contra um espaço teórico de $2^{10} = 1024$ subconjuntos.

3.2. Branch and Bound

O Branch and Bound explora a árvore de decisão em largura (BFS) usando uma fila explícita de estados. Antes de expandir cada estado, calcula um limite superior (upper bound) usando somas de sufixo pre-computadas. Se o upper bound for insuficiente, o estado é descartado.

Pre-computo das somas de sufixo ($O(n)$ uma vez):

```
suffix_sum[i] = nums[i] + nums[i+1] + ... + nums[n-1]
```

Limite superior de qualquer estado (index, current_sum) calculado em $O(1)$:

```
upper_bound = current_sum + suffix_sum[index]
```

3.2.1. Análise de Corretude

O Branch and Bound é correto e completo pelos mesmos motivos do Backtracking:

- Explora toda a árvore de decisão (no pior caso).
- As podas são conservadoras: descartam apenas estados onde o `upper_bound < target` (provavelmente sem solução).
- O BFS garante que, ao encontrar uma solução, ela é a primeira atingida naquele nível de profundidade.

A única diferença em relação ao Backtracking está na solução específica retornada quando há múltiplas soluções válidas, não na garantia de encontrá-la.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

O B&B mantém uma fila de estados. No pior caso, todos os estados de um nível da árvore ficam simultaneamente na fila.

Caso	Complexidade de Espaço
Pior caso	$O(2^n)$ — fila pode conter todos os estados de um nível
Caso favorável (podas eficazes)	$O(1)$ a $O(n)$ — fila permanece pequena
Memória observada nos testes	0,74 KB (sem solução) a 14,10 KB (fila_grande)

A memória é o grande ponto fraco do B&B: cresce proporcionalmente ao número de caminhos viáveis simultâneos. No cenário `fila_grande_bb`, a fila atingiu 127 estados simultâneos, consumindo 14,10 KB — 34x mais que o Backtracking.

3.2.2.2. Temporal

Caso	Complexidade de Tempo	Situacao
Pior caso	$O(2^n)$	Sem podas — todos os estados enfileirados
Caso favoravel	Melhor que Backtracking	Podas pelo <code>upper_bound</code> são mais precisas
Pior cenário observado	0,57 ms (<code>fila_grande</code> , $n=15$)	Muitos caminhos viáveis — poda ineficaz

O cálculo do `upper_bound` em $O(1)$ via `suffix_sum` e a principal vantagem. O overhead e o custo de manter e copiar a lista de selecionados para cada estado na fila.

3.3. Programacao Dinamica

A Programação Dinâmica (DP) resolve o problema pela construção bottom-up de uma tabela booleana `dp[i][s]`, onde `dp[i][s] = True` se existe subconjunto dos primeiros i elementos cuja soma é s .

Recorrencia:

```
dp[0][0] = True                # subconjunto vazio soma 0
dp[0][s] = False para s > 0

dp[i][s] = dp[i-1][s]          # nao inclui nums[i-1]
          OR dp[i-1][s - nums[i-1]] # inclui nums[i-1], se s >= nums[i-1]
```

Após preencher a tabela, o subconjunto é reconstruído fazendo backtracking sobre `dp[n][target]` ate `dp[0][0]`.

3.3.1. Análise de Corretude

A DP é correta e completa por construção:

- A recorrência cobre todos os casos: incluir ou não cada elemento.
- `dp[n][target] = True` se e somente se existe solução — provável por indução sobre i .
- A reconstrução é determinística: segue o caminho oposto ao preenchimento, identificando exatamente quais elementos foram incluídos.
- Não há heurística nem poda: a tabela é sempre completamente preenchida.

A DP é o único algoritmo deste projeto que garante encontrar a solução sem explorar o espaço de estados, mas sim resolvendo subproblemas menores de forma sistemática.

3.3.2. Análise de Desempenho

3.3.2.1. Espacial

A tabela DP tem dimensões $(n+1) \times (T+1)$. Cada célula é um booleano (1 byte em Python representado internamente como objeto).

Caso	Complexidade de Espaço
Todos os casos	$O(n \times T)$ — tabela sempre completamente alocada
Otimização possível	$O(T)$ — usando apenas 1 linha (sem reconstrução)
Memória observada nos testes	0,63 KB ($n=4$, $T=9$) a 8,55 KB ($n=12$, $T=79$)

Ponto crítico: a memória cresce com T , não só com n . Para $T = 10^6$ e $n = 100$, a tabela tem 10^8 células — inviável. A DP é pseudopolinomial: polinomial em n e T , mas T pode ser exponencial em relação ao número de bits de entrada.

3.3.2.2. Temporal

Caso	Complexidade de Tempo	Observação
Todos os casos	$O(n \times T)$	Sem variação — tabela sempre preenchida por completo
Melhor caso = pior caso	$O(n \times T)$	DP não tem poda nem early-exit
Comparativo nos testes	Mais lento para T grande	sem_solucao: 0,81 ms (maior $T = 79$)

A DP é o algoritmo com comportamento mais previsível: o tempo é exatamente $O(n \times T)$ em qualquer instância. Para T pequeno (cenários com $T < 30$), é competitivo ou melhor que Backtracking e B&B. Para T grande, pode ser o mais lento.

3.4. Estratégia Gulosa

A Estratégia Gulosa ordena os elementos em ordem decrescente e, a cada passo, inclui o maior elemento que não ultrapasse o valor restante.

Critério guloso: "escolha sempre o maior elemento que ainda cabe no espaço restante".

Pseudocódigo:

```
ordenar indices por nums[i] decrescente
remaining = target
para cada idx na ordem decrescente:
    se nums[idx] <= remaining:
        incluir nums[idx]
        remaining -= nums[idx]
    senao:
        descartar (poda)
se remaining == 0: parar
```

3.4.1. Análise de Corretude

A Estratégia Gulosa NAO e correta nem completa para o Subset Sum:

- Não garante encontrar solução quando ela existe (falso negativo possível).
- Quando encontra uma solução, ela é válida (não há falso positivo).

Exemplo clássico de falha:

```
Conjunto = {10, 7, 6, 5, 3}, Alvo = 11
Guloso: escolhe 10 (remaining = 1) -> nenhum elemento cabe -> FALHA
Solucao real: {6, 5} -> 6 + 5 = 11 (EXISTS)
```

Nos testes, o Guloso falhou em 2 de 7 cenários (input.txt e poda_maxima.txt), confirmando sua natureza aproximada.

Por que ainda é útil? Para instâncias onde a solução "gulosa" existe (valores bem distribuídos), o Guloso é dramaticamente mais rápido que as demais técnicas, com complexidade $O(n \log n)$.

3.4.2. Análise de Desempenho

3.4.2.1. Espacial

Caso	Complexidade de Espaço
Todos os casos	$O(n)$ — array de índices ordenados + lista de selecionados
Memória observada nos testes	0,30 a 0,39 KB (constante na prática)

O Guloso tem a menor memória entre os quatro algoritmos. Não aloca tabelas nem filas — apenas arrays auxiliares de tamanho n .

3.4.2.2. Temporal

Caso	Complexidade de Tempo	Observacao
Ordenacao inicial	$O(n \log n)$	Dominante
Varredura gulosa	$O(n)$	Passagem única pelos elementos ordenados
Total	$O(n \log n)$	O mais rápido dos quatro algoritmos
Tempo observado nos testes	0,008 a 0,013 ms	Consistentemente o mais rápido

O Guloso é entre 3x e 50x mais rápido que os demais algoritmos nos testes realizados. O trade-off é a falta de garantia de encontrar a solução.

4. Experimentos e Resultados

4.1. Instancias de Teste

Foram projetados 7 cenários para expor características distintas de cada algoritmo:

#	Arquivo	n	T	Conjunto	Objetivo
1	input.txt	4	9	{3, 4, 5, 8}	Caso base — verificação de corretude
2	padrao.txt	8	15	{3,5,7,2,8,1,4,6}	Caso equilibrado — referencia
3	solucao_imediata.txt	10	12	{5,7,9,3,8,2,6,4,1,10}	Solução nos 2 primeiros elementos
4	sem_solucao.txt	12	79	{1..12}	Sem solução — soma total (78) < T
5	poda_maximal.txt	10	5	{20,18,15,...,2}	Elementos grandes, alvo pequeno
6	solucao_no_final.txt	10	9	{4x9, 9}	Solucao apenas no último elemento
7	fila_grande_bb.txt	15	30	{2,3,...,16}	Muitos caminhos viáveis

4.2. Resultados Obtidos

Cenário 1 — input.txt (n=4, T=9)

Metrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {4,5}	SIM — {4,5}	SIM — {4,5}	NAO (falha gulosa)
Nos visitados / células	13	11	41	4
Podas / rejeitados	4	8	0	3
Eficiencia de poda	23,5%	42,1%	N/A	42,9%
Prof. max / Fila max / Tabela	4	6	50 células	—
Tempo (ms)	0,0564	0,0656	0,0306	0,0101
Memoria (KB)	0,30	1,62	0,63	0,30
Qualidade	Exata	Exata	Exata	FALHA

Obs.: Guloso falhou: escolheu 8 (remaining=1), sem elemento de valor 1 disponível para completar.

Cenário 2 — padrao.txt (n=8, T=15) — Caso Base

Métrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {3,5,7}	SIM — {3,5,7}	SIM — {3,5,7}	SIM — {7,8}
Nos visitados / celulas	4	8	129	2
Podas / rejeitados	0	0	0	0
Eficiencia de poda	0,0%	0,0%	N/A	0,0%
Prof. max / Fila max / Tabela	3	8	144 celulas	—
Tempo (ms)	0,0112	0,0250	0,0493	0,0085
Memoria (KB)	0,28	1,12	1,54	0,33
Qualidade	Exata	Exata	Exata	Exata (aprox.)

Obs.: Guloso encontrou {7,8} em vez de {3,5,7} — ambas válidas. BT encontrou com apenas 4 nós. DP preencheu 129 células desnecessariamente.

Cenário 3 — solucao_imediata.txt (n=10, T=12) — Vantagem BT/BB

Métrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {5,7}	SIM — {5,7}	SIM — {5,7}	SIM — {2,10}
Nos visitados / celulas	3	4	131	9
Podas / rejeitados	0	0	0	7
Eficiencia de poda	0,0%	0,0%	N/A	43,8%
Prof. max / Fila max / Tabela	2	4	143 celulas	—
Tempo (ms)	0,0085	0,0097	0,0539	0,0094
Memoria (KB)	0,38	0,78	1,65	0,34
Qualidade	Exata	Exata	Exata	Exata (aprox.)

Obs.: BT visitou apenas 3 nós — solução nos 2 primeiros elementos. DP preencheu 131 células sem nenhuma poda. Guloso encontrou uma solução diferente ({2,10}) mas igualmente válida.

Cenário 4 — sem_solucao.txt (n=12, T=79) — Sem Solução

Métrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	NAO	NAO	NAO	NAO
Nos visitados / celulas	1	1	961	12
Podas / rejeitados	1	1	0	0
Eficiencia de poda	50,0%	50,0%	N/A	0,0%
Prof. max / Fila max / Tabela	0	1	1040 celulas	—
Tempo (ms)	0,0045	0,0027	0,8091	0,0118
Memoria (KB)	0,37	0,74	8,55	0,36

Qualidade	Exata (prova impossib.)	Exata (prova impossib.)	Exata (prova impossib.)	Inconclusivo
-----------	-------------------------	-------------------------	-------------------------	--------------

Obs.: BT e BB detectaram a impossibilidade com 1 no, via poda (soma total $78 < T=79$). DP preencheu 961 células e usou 8,55 KB — o pior resultado absoluto. Guloso percorreu todos os 12 elementos sem conclusão definitiva.

Cenário 5 — poda_maxima.txt (n=10, T=5) — Maxima Poda

Métrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {3,2}	SIM — {3,2}	SIM — {3,2}	NAO (falha gulosa)
Nos visitados / celulas	14	13	61	10
Podas / rejeitados	9	12	0	9
Eficiencia de poda	39,1%	48,0%	N/A	47,4%
Prof. max / Fila max / Tabela	10	2	66 celulas	—
Tempo (ms)	0,0304	0,0146	0,0404	0,0082
Memoria (KB)	0,35	0,77	0,94	0,34
Qualidade	Exata	Exata	Exata	FALHA

Obs.: Elementos grandes (20,18,...) com alvo pequeno (5). Poda por excesso é extremamente eficaz: BB atingiu 48% de eficiência. Guloso falhou: escolheu 4 (remaining=1) sem ter elemento de valor 1.

Cenário 6 — solucao_no_final.txt (n=10, T=9) — Destaque da DP

Métrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {9}	SIM — {9}	SIM — {9}	SIM — {9}
Nos visitados / celulas	221	176	101	1
Podas / rejeitados	129	175	0	0
Eficiencia de poda	36,9%	49,9%	N/A	0,0%
Prof. max / Fila max / Tabela	10	46	110 celulas	—
Tempo (ms)	0,3567	0,3922	0,0717	0,0113
Memoria (KB)	0,38	6,23	1,17	0,34
Qualidade	Exata	Exata	Exata	Exata (aprox.)

Obs.: Conjunto {4x9, 9}: solução é o último elemento. BT explorou 221 nós, BB acumulou fila de 46 estados. DP preencheu apenas 101 celulas e foi o mais rápido dos exatos (0,07 ms). Guloso acertou de imediato (primeiro elemento de valor $9 \leq 9$).

Cenário 7 — fila_grande_bb.txt (n=15, T=30) — Ponto Fraco do BB

Metrica	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Solucao encontrada	SIM — {2,3,4,5,6,10}	SIM — {2,3,4,6,7,8}	SIM — {2,3,4,6,7,8}	SIM — {14,16}
Nos visitados / celulas	37	135	466	3
Podas / rejeitados	24	8	0	1
Eficiencia de poda	39,3%	5,6%	N/A	25,0%
Prof. max / Fila max / Tabela	15	127	496 celulas	—
Tempo (ms)	0,1761	0,6948	0,4296	0,0128
Memoria (KB)	0,41	14,16	4,19	0,39
Qualidade	Exata	Exata	Exata	Exata (aprox.)

Obs.: Muitos caminhos viáveis simultâneos: BB acumulou fila de 127 estados (14,16 KB) e foi o mais lento dos exatos (0,69 ms). BT foi 3,9x mais rápido. DP preencheu 466 células mas usou apenas 4,19 KB. Guloso: 0,013 ms com solução válida.

Tabela Consolidada — Todos os Cenários e Algoritmos

Cenario	Alg.	Nos/Celulas	Podas	Efic.Poda	Tempo(ms)	Mem(KB)	Solucao
input (n=4)	BT	13	4	23,5%	0,0564	0,30	SIM
input (n=4)	BB	11	8	42,1%	0,0656	1,62	SIM
input (n=4)	DP	41	0	N/A	0,0306	0,63	SIM
input (n=4)	GR	4	3	42,9%	0,0101	0,30	FALHA
padrao (n=8)	BT	4	0	0,0%	0,0112	0,28	SIM
padrao (n=8)	BB	8	0	0,0%	0,0250	1,12	SIM
padrao (n=8)	DP	129	0	N/A	0,0493	1,54	SIM
padrao (n=8)	GR	2	0	0,0%	0,0085	0,33	SIM
sol_imed. (n=10)	BT	3	0	0,0%	0,0085	0,38	SIM
sol_imed. (n=10)	BB	4	0	0,0%	0,0097	0,78	SIM
sol_imed. (n=10)	DP	131	0	N/A	0,0539	1,65	SIM
sol_imed. (n=10)	GR	9	7	43,8%	0,0094	0,34	SIM
sem_sol. (n=12)	BT	1	1	50,0%	0,0045	0,37	NAO
sem_sol. (n=12)	BB	1	1	50,0%	0,0027	0,74	NAO
sem_sol. (n=12)	DP	961	0	N/A	0,8091	8,55	NAO

sem_sol. (n=12)	GR	12	0	0,0%	0,0118	0,36	NAO(inc)
poda_max. (n=10)	BT	14	9	39,1%	0,0304	0,35	SIM
poda_max. (n=10)	BB	13	12	48,0%	0,0146	0,77	SIM
poda_max. (n=10)	DP	61	0	N/A	0,0404	0,94	SIM
poda_max. (n=10)	GR	10	9	47,4%	0,0082	0,34	FALHA
sol_final (n=10)	BT	221	129	36,9%	0,3567	0,38	SIM
sol_final (n=10)	BB	176	175	49,9%	0,3922	6,23	SIM
sol_final (n=10)	DP	101	0	N/A	0,0717	1,17	SIM
sol_final (n=10)	GR	1	0	0,0%	0,0113	0,34	SIM
fila_grd. (n=15)	BT	37	24	39,3%	0,1761	0,41	SIM
fila_grd. (n=15)	BB	135	8	5,6%	0,6948	14,16	SIM
fila_grd. (n=15)	DP	466	0	N/A	0,4296	4,19	SIM
fila_grd. (n=15)	GR	3	1	25,0%	0,0128	0,39	SIM

5. Análise e Discussão

5.1. Comparacao das Estrategias

Complexidade assintótica comparada:

Critério	Backtracking	Branch & Bound	Prog. Dinamica	Est. Gulosa
Complexidade tempo	$O(2^n)$	$O(2^n)$	$O(n \times T)$	$O(n \log n)$
Complexidade espaco	$O(n)$	$O(2^n)$	$O(n \times T)$	$O(n)$
Estrategia de busca	DFS	BFS	Bottom-up DP	Gulosa
Garante solução exata	SIM	SIM	SIM	NAO
Memoria constante	SIM	NAO	NAO	SIM
Depende do valor de T	NAO	NAO	SIM	NAO

Vitorias por critério nos testes:

Criterio	Vencedor	Detalhe
Mais rápido (em média)	Gulosa	$O(n \log n)$ — sempre abaixo de 0,013 ms
Mais rápido entre exatos	Backtracking	5 de 7 cenários — DFS encontra rápido
Menor memória	BT / Gulosa	Ambos $O(n)$ — BT: 0,28-0,41 KB; GR: 0,30-0,39 KB
Maior eficiência de poda	Branch & Bound	4 de 7 cenários
Melhor no sem_solucao	B&B	0,0027 ms com 1 nó visitado
Melhor no sol_no_final	Prog. Dinamica	0,0717 ms — BT usou 221 nós, BB 176
Pior caso de memória	B&B	fila_grande: 14,16 KB
Mais previsível	Prog. Dinamica	Sempre $O(n \times T)$, sem variação

Quando usar cada algoritmo:

Situacao	Algoritmo Recomendado	Motivo
Memória limitada	Backtracking	$O(n)$ garantido em qualquer cenário
T pequeno, n grande	Prog. Dinamica	$O(n \times T)$ e competitivo quando $T \ll 2^n$
Poda eficaz esperada	Branch & Bound	Upper_bound preciso elimina estados cedo
Velocidade acima de tudo	Gulosa	$O(n \log n)$ — aceitar risco de falha
Muitos caminhos viáveis	Backtracking	BB sofre com fila crescente
Solução no início do array	Backtracking	DFS encontra imediatamente
Necessidade de prova de impossibilidade	BT / BB / DP	Gulosa e inconclusiva

6. Conclusão

Este trabalho implementou e comparou quatro técnicas de projeto de algoritmos aplicadas ao Problema do Subconjunto Soma, um problema NP-completo clássico. Os resultados experimentais revelam um panorama rico de trade-offs.

Backtracking e Branch and Bound são ambos exatos e exploram o mesmo espaço de busca em estratégias opostas (DFS vs BFS). O Backtracking é superior em uso de memória ($O(n)$ sempre) e em cenários onde a solução está no início do espaço DFS. O Branch and Bound apresenta eficiência de poda superior graças ao cálculo preciso do upper bound via suffix_sum, sendo mais rápido em cenários de poda intensa, mas podendo ser drasticamente mais lento e maior em memória quando há muitos caminhos viáveis.

A Programação Dinâmica é o algoritmo mais previsível: sempre $O(n \times T)$ em tempo e espaço, sem variação. É o melhor exato para cenários onde a solução está dispersa no espaço de busca (como `sol_no_final`). Seu ponto fraco é a dependência do valor de T : para T grande, pode ser o mais lento e o que mais consome memória.

A Estratégia Gulosa é dramaticamente mais rápida que todos os demais, com $O(n \log n)$, mas paga o preço de não garantir solução. Em 2 dos 7 cenários testados, falhou completamente. Sua utilidade é máxima quando a instância favorece a escolha gulosa, ou quando uma solução aproximada é aceitável em troca de velocidade.

Conclusão geral: não há algoritmo universalmente superior para o Subset Sum. A escolha ótima depende do perfil da instância — tamanho de n , magnitude de T , distribuição dos elementos e requisitos de exatidão. O conhecimento das características de cada técnica é fundamental para a decisão.

7. Referencias

- CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. MIT Press, 2022.
- KLEINBERG, J.; TARDOS, E. Algorithm Design. Addison-Wesley, 2005.
- GAREY, M. R.; JOHNSON, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, 1979.
- SIPSER, M. Introduction to the Theory of Computation. 3. ed. Cengage Learning, 2012.
- Python Software Foundation. Python 3.11 Documentation — time, tracemalloc. Disponível em: <https://docs.python.org/3/>